

</2. Algoritmos de Ordenamiento

Definición:

Los algoritmos de ordenamiento son procesos o conjuntos de instrucciones diseñados para reorganizar los elementos de una estructura de datos (como un arreglo o lista) en un orden específico, generalmente **ascendente** o **descendente**.

Clasificación:

- **Internos:** Ordenan los datos que caben completamente en la memoria principal (RAM).
- **Externos:** Diseñados para manejar grandes volúmenes de datos que no caben en la memoria principal, usando almacenamiento secundario (Disco).

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Importancia y uso de los algoritmos de ordenamiento:

Razones para usar algoritmos de ordenamiento:

- **Eficiencia:** Mejoran el rendimiento de búsquedas (por ejemplo, las búsquedas binarias requieren datos ordenados).
- **Organización:** Facilitan la presentación de información de manera lógica.
- **Optimización:** Son la base para operaciones avanzadas, como algoritmos de fusión o análisis de datos.

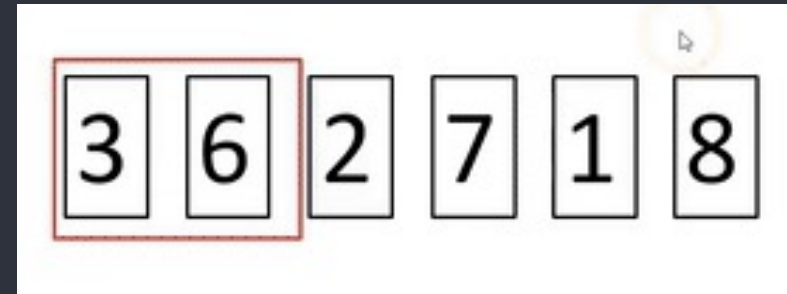
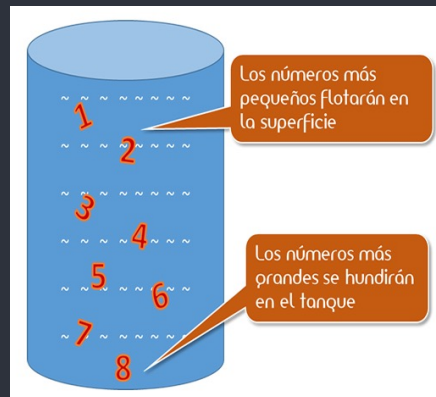
Áreas de aplicación:

- **Bases de datos:** Clasificación de registros.
- **Análisis de datos:** Preparación de conjuntos de datos.
- **Redes:** Organización de paquetes de datos.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ordenamiento por el Método Burbuja:

Es el algoritmo de ordenamiento más sencillo que existe, al mismo tiempo es poco eficiente. Funciona comparando dos elementos de una lista de datos y los intercambia de posición si están en el orden equivocado. Revisa varias veces toda la lista hasta que no se necesiten más intercambios, lo cual significa que la lista está ordenada.



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Pseudocódigo:



```
Repetir para i desde 0 hasta n-2:  
  Repetir para j desde 0 hasta n-i-2:  
    Si arreglo[j] > arreglo[j+1]:  
      Intercambiar arreglo[j] y arreglo[j+1]
```

Complejidad temporal:

- **Mejor caso:** $O(n)$ cuando la lista ya está ordenada, con optimización.
- **Peor caso:** $O(n^2)$.
- **Caso promedio:** $O(n^2)$.

Ventajas:

- Fácil de entender e implementar.
- Útil para listas pequeñas.

Desventajas:

- Muy ineficiente para listas grandes debido a su alta complejidad.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ordenamiento por Selección:

El método de selección encuentra el elemento más pequeño (o grande) en cada iteración y lo coloca en su posición correcta.

8
5
2
6
9
3
1
4
0
7



```
Para i desde 0 hasta n-1:
    MinIndex = i
    Para j desde i+1 hasta n-1:
        Si arreglo[j] < arreglo[MinIndex]:
            MinIndex = j
    Intercambiar arreglo[i] y arreglo[MinIndex]
```

Complejidad temporal:

- Mejor caso: $O(n^2)$.
- Peor caso: $O(n^2)$.
- Caso promedio: $O(n^2)$.

Ventajas:

- Realiza menos intercambios que el método burbuja.
- Simple de entender e implementar.

Desventajas:

- Ineficiente para listas grandes.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Comparación entre los métodos burbuja y selección:

Aspecto	Método Burbuja	Método Selección
Iteraciones	Más iteraciones e intercambios.	Menos intercambios.
Simplicidad	Fácil de entender.	Igual de fácil de implementar.
Eficiencia	Menos eficiente en promedio.	Ligeramente mejor en promedio.
Complejidad	$O(n^2)$ en todos los casos.	$O(n^2)$ en todos los casos.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Complejidad temporal:

La complejidad temporal mide el tiempo que tarda un algoritmo en ejecutarse en función del tamaño de la entrada. Se utiliza para analizar cómo crece el tiempo de ejecución de un programa conforme aumenta el tamaño de los datos que procesa. Este análisis se realiza de manera abstracta, sin depender de la máquina en la que se ejecuta el programa o de los detalles específicos de implementación.

¿Por qué es importante la complejidad temporal?

1. **Comparar eficiencia:** Ayuda a determinar qué algoritmo es más eficiente para un problema dado, especialmente cuando se trabaja con grandes volúmenes de datos.
2. **Predecir escalabilidad:** Permite prever si un algoritmo será práctico cuando el tamaño de la entrada crezca.
3. **Optimización:** Identificar algoritmos que son rápidos y evitar aquellos que consumen demasiado tiempo.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Cómo se expresa la complejidad temporal:

La complejidad temporal se representa usando la **notación Big-O ($O(\cdot)$)**, que describe el **crecimiento del tiempo de ejecución** en el peor caso, mejor caso o caso promedio, en términos del tamaño de la entrada (n).

1. **Big-O (O)**: Describe el peor caso.
2. **Big- Ω (Ω)**: Describe el mejor caso.
3. **Big- Θ (Θ)**: Describe el caso promedio o tiempo exacto.

Ejemplo básico de complejidad temporal: Buscar un elemento en un arreglo.

1. Búsqueda lineal:

- Recorres todos los elementos de un arreglo uno por uno.
- Complejidad: $O(n)$, porque, en el peor caso, tendrás que revisar todos los elementos para encontrar el que buscas.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

2. Búsqueda binaria:

- Funciona en arreglos ordenados, dividiendo el rango de búsqueda a la mitad en cada paso.
- Complejidad: $O(\log n)$, ya que cada paso reduce el tamaño del problema a la mitad.

Clases comunes de complejidad temporal:

1. Constante ($O(1)$):

- Tiempo de ejecución fijo, independientemente del tamaño de la entrada.
- Ejemplo: Acceder a un elemento específico de un arreglo.

2. Lineal ($O(n)$):

- Tiempo de ejecución crece proporcionalmente al tamaño de la entrada.
- Ejemplo: Recorrer todos los elementos de una lista.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

3. Cuadrática ($O(n^2)$):

- Tiempo de ejecución crece al cuadrado del tamaño de la entrada.
- Ejemplo: Algoritmos de ordenamiento como burbuja o selección.

4. Logarítmica ($O(\log n)$):

- Crecimiento mucho más lento que lineal.
- Ejemplo: Búsqueda binaria.

5. Lineal-logarítmica ($O(n \log n)$):

- Más eficiente que cuadrática pero menos que lineal.
- Ejemplo: Algoritmos de ordenamiento eficientes como mergesort.

6. Exponencial ($O(2^n)$):

- El tiempo de ejecución crece exponencialmente con el tamaño de la entrada.
- Ejemplo: Fuerza bruta para resolver problemas complejos.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ilustración de cómo crecen las diferentes complejidades:

Si tienes un algoritmo que procesa $n = 10$ elementos, este es el número aproximado de pasos que realizará según la complejidad:

Complejidad	Pasos para $n = 10$	Pasos para $n = 100$
$O(1)$	1	1
$O(\log n)$	3	7
$O(n)$	10	100
$O(n \log n)$	30	700
$O(n^2)$	100	10,000
$O(2^n)$	1,024	Inviabile

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ejemplo práctico: Recorrer un arreglo.

```

#include <iostream>
using namespace std;

// Función para recorrer y mostrar el arreglo
void recorrerArreglo(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    // Declarar un arreglo con 5 elementos
    int arr[] = {1, 2, 3, 4, 5};

    // Especificar explícitamente el tamaño del arreglo
    int n = 5;

    // Llamar a la función con el arreglo y su tamaño
    recorrerArreglo(arr, n);

    return 0;
}
```

- **Entrada:** 5 elementos.
- **Salida:** Se imprime cada elemento una vez.
- **Complejidad:** $O(n)$, porque cada elemento se procesa una vez.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

¿Cómo saber la complejidad de un algoritmo?

Si tienes un algoritmo que procesa $n = 10$ elementos, este es el número aproximado de pasos que realizará según la complejidad:

1. Identifica los bucles:

- Un bucle simple suele contribuir con $O(n)$.
- Bucles anidados (uno dentro de otro) generan $O(n^2)$.

2. Considera llamadas recursivas:

- Si el algoritmo usa recursión, analiza cuántas veces se llama a sí mismo y con qué tamaño de entrada.

3. Busca operaciones clave:

- Acceso a estructuras de datos: ¿Son operaciones rápidas como en un arreglo ($O(1)$) o más lentas como en una lista enlazada ($O(n)$)?

4. Evalúa el peor caso:

- Considera la entrada que lleva más tiempo procesar.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ejercicios:

1. Dado un arreglo de números enteros, ordénalo de menor a mayor utilizando el método burbuja o selección.
2. Dado un arreglo de cadenas que representan nombres de estudiantes, ordénalos alfabéticamente (de A a Z) utilizando el método de selección.
3. Dado un arreglo de precios de productos, ordénalos de mayor a menor utilizando el método burbuja.
4. Dado un arreglo de números y un entero k, utiliza el método de selección para encontrar los k menores elementos del arreglo.
5. Dada una matriz (arreglo bidimensional), ordena cada fila de manera ascendente utilizando el método burbuja o selección.
6. Escribe un programa que permita al usuario ingresar dos arreglos de números enteros. El programa debe fusionar ambos arreglos en uno solo y ordenarlo en orden ascendente utilizando el método de selección.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ordenamiento por Inserción:

El ordenamiento por inserción (Insertion Sort) es un algoritmo sencillo que organiza una lista iterando de izquierda a derecha. Cada elemento se extrae y se inserta en la posición correcta dentro de la parte ordenada del arreglo. Es útil para listas pequeñas o casi ordenadas.

Este algoritmo funciona de manera similar a como los humanos organizan cartas en la mano. Es un algoritmo in situ (no requiere memoria adicional significativa) y estable (no altera el orden relativo de los elementos iguales).

Ventajas:

- Simple de implementar.
- Eficiente para arreglos pequeños o casi ordenados.

Desventajas:

- Ineficiente para listas grandes debido a su complejidad temporal en el peor caso.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Pseudocódigo:

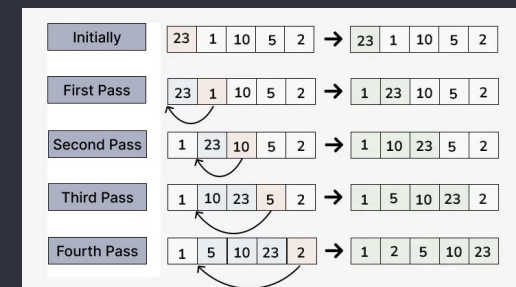
```
Para i desde 1 hasta n-1:  
  key = arreglo[i]  
  j = i - 1  
  Mientras j >= 0 y arreglo[j] > key:  
    arreglo[j+1] = arreglo[j]  
    j = j - 1  
  arreglo[j+1] = key
```

6 5 3 1 8 7 2 4

Ejemplo:

Arreglo inicial: [5, 3, 4, 1, 2]

1. Primera iteración: 5 ya está ordenado.
2. Segunda iteración: Se inserta 3 en la posición correcta: [3, 5, 4, 1, 2]
3. Tercera iteración: Se inserta 4: [3, 4, 5, 1, 2]
4. Cuarta iteración: Se inserta 1: [1, 3, 4, 5, 2]
5. Quinta iteración: Se inserta 2: [1, 2, 3, 4, 5]



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Complejidad Temporal:

La complejidad temporal de Insertion Sort depende del caso:

- **Mejor caso:** $O(n)$ ocurre cuando el arreglo ya está ordenado. Solo realiza $n-1$ comparaciones y ninguna asignación adicional.
- **Caso promedio:** $O(n^2)$ en el que los elementos están distribuidos al azar y se hacen $n^2/4$ operaciones en promedio.
- **Peor caso:** $O(n^2)$ cuando el arreglo está completamente ordenado de manera inversa. En este caso, se hacen $n^2/2$ operaciones.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Quick Sort:

Quick Sort es un algoritmo de ordenamiento **divide y vencerás** que selecciona un elemento como pivote y particiona el arreglo en dos subarreglos: los elementos menores que el pivote y los elementos mayores. Luego aplica el mismo proceso recursivamente.

El propósito del pivote es actuar como referencia para reorganizar los elementos en el arreglo durante el proceso de partición.

Ventajas:

- Muy eficiente en promedio.
- Utilizado en muchas bibliotecas estándar debido a su rapidez.

Desventajas:

- No es estable porque durante el proceso de partición los elementos con valores iguales al pivote pueden cambiar su orden relativo. Esto ocurre porque el algoritmo típicamente intercambia elementos sin preocuparse por mantener su posición original.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

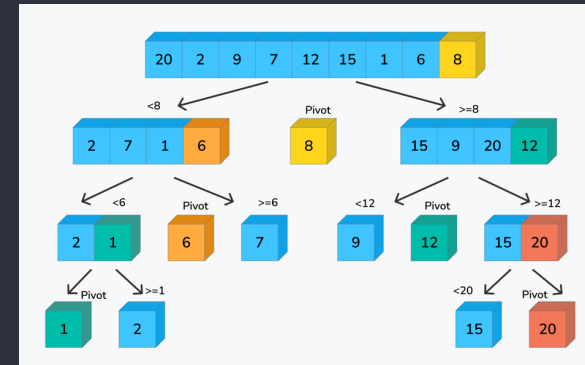
Pseudocódigo:

```
QuickSort(arreglo, inicio, fin):  
  Si inicio < fin:  
    pivote = Particionar(arreglo, inicio, fin)  
    QuickSort(arreglo, inicio, pivote - 1)  
    QuickSort(arreglo, pivote + 1, fin)  
  
Particionar(arreglo, inicio, fin):  
  pivote = arreglo[fin]  
  i = inicio - 1  
  Para j desde inicio hasta fin-1:  
    Si arreglo[j] <= pivote:  
      i = i + 1  
      Intercambiar(arreglo[i], arreglo[j])  
  Intercambiar(arreglo[i+1], arreglo[fin])  
  Retornar i + 1
```

Ejemplo:

Arreglo inicial: [8, 4, 7, 3, 5]

1. Seleccionar pivote (5).
2. Reorganizar elementos: [4, 3, 5, 8, 7].
3. Llamar recursivamente a Quick Sort para subarreglos: [4, 3] y [8, 7].
4. Resultado final: [3, 4, 5, 7, 8].



1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Complejidad Temporal:

- **Mejor caso:** $O(n \log n)$. Esto ocurre cuando el pivote divide el arreglo en dos partes aproximadamente iguales en cada nivel de partición. Este equilibrio maximiza la eficiencia.
- **Caso promedio:** $O(n \log n)$. En promedio, los arreglos son divididos de manera razonablemente balanceada en cada partición, lo que resulta en un desempeño eficiente.
- **Peor caso:** $O(n^2)$. Sucede cuando el pivote es siempre el menor o el mayor elemento, lo que genera particiones altamente desbalanceadas (por ejemplo, un arreglo ya ordenado o en orden inverso con una mala selección del pivote).

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Merge Sort:

Merge Sort es un algoritmo de ordenamiento recursivo basado en divide y vencerás. Divide repetidamente el arreglo en mitades hasta que los subarreglos tengan un elemento, y luego los combina en orden.

Ventajas:

- Estable y eficiente.
- Ideal para listas grandes y datos almacenados en discos.

Desventajas:

- Requiere memoria adicional proporcional al tamaño del arreglo.

5	3	7	1	0	8	5
---	---	---	---	---	---	---

`mergesort([5, 3, 7, 1, 0, 8, 5])`

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Pseudocódigo:

```
MergeSort(arreglo):
  Si longitud(arreglo) > 1:
    medio = longitud(arreglo) // 2
    izquierda = arreglo[0:medio]
    derecha = arreglo[medio:]

    MergeSort(izquierda)
    MergeSort(derecha)

    Merge(arreglo, izquierda, derecha)

Merge(arreglo, izquierda, derecha):
  i = j = k = 0
  Mientras i < longitud(izquierda) y j < longitud(derecha):
    Si izquierda[i] < derecha[j]:
      arreglo[k] = izquierda[i]
      i += 1
    Sino:
      arreglo[k] = derecha[j]
      j += 1
    k += 1
  Copiar elementos restantes de izquierda y derecha si los hay.
```

Ejemplo:

Arreglo inicial: [8, 4, 7, 3, 5]

1. Dividir en [8, 4, 7] y [3, 5].
2. Dividir nuevamente: [8, 4], [7], [3], [5].
3. Combinar: [4, 8], [3, 5].
4. Combinar final: [3, 4, 5, 7, 8].

<https://visualgo.net/>

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Complejidad Temporal:

- Mejor caso: $O(n \log n)$
- Peor caso: $O(n \log n)$
- Caso promedio: $O(n \log n)$

Esto se debe a que Merge Sort siempre divide el conjunto de datos a la mitad en cada paso (lo que toma $O(\log n)$) y luego fusiona las partes de manera ordenada, lo que requiere $O(n)$ tiempo por cada nivel de recursión. Dado que la recursión ocurre $\log n$ veces (en la mayoría de los casos), la complejidad total es $O(n \log n)$.

A diferencia de otros algoritmos como Quick Sort, que pueden tener un peor caso de $O(n^2)$, Merge Sort tiene un rendimiento predecible, garantizando siempre $O(n \log n)$, lo que lo hace especialmente útil cuando se necesita una ordenación estable y eficiente.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Comparación de algoritmos:

Característica	Insertion Sort	Quick Sort	Merge Sort
Enfoque	Ordenamiento incremental, construye la lista ordenada paso a paso.	Divide y vencerás con un pivote.	Divide y vencerás, divide en mitades y fusiona.
Complejidad Temporal	Mejor: $O(n)$ Promedio: $O(n^2)$ Peor: $O(n^2)$	Mejor y promedio: $O(n \log n)$ Peor: $O(n^2)$	Siempre $O(n \log n)$
Estabilidad	Es estable.	No es estable.	Es estable.
Rendimiento en la práctica	Lento para grandes volúmenes de datos.	Rápido en la mayoría de los casos.	Más lento que Quick Sort en la práctica debido al uso de memoria adicional.
Casos de uso típicos	Pequeños conjuntos de datos o cuando los datos ya están casi ordenados.	Datos grandes o cuando se busca velocidad en promedio.	Datos grandes donde se necesita una complejidad garantizada de $O(n \log n)$
Recursividad	No es recursivo.	Recursivo.	Recursivo.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Conclusiones:

- **Insertion Sort:** Mejor para conjuntos de datos pequeños o parcialmente ordenados debido a su simplicidad, pero no es adecuado para grandes cantidades de datos.
- **Quick Sort:** Es generalmente el más rápido en la práctica para datos grandes, pero su rendimiento puede empeorar en el peor caso si no se implementa correctamente (por ejemplo, si el pivote no se selecciona adecuadamente).
- **Merge Sort:** Siempre ofrece un rendimiento garantizado de $O(n \log n)$, es estable, y es ideal cuando se necesitan ordenamientos más predecibles o cuando se trabaja con grandes volúmenes de datos, aunque su uso de memoria puede ser un inconveniente en ciertos casos.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Ejercicios:

1. Dado un arreglo de enteros desordenado, se debe implementar el algoritmo de ordenamiento Insertion Sort para organizar los elementos de menor a mayor.
2. Dado un arreglo de enteros desordenado, implementa el algoritmo de ordenamiento Quick Sort para ordenar el arreglo de menor a mayor.
3. Dado un arreglo de enteros desordenado, implementa el algoritmo Merge Sort para ordenar el arreglo de menor a mayor.
4. Dado un arreglo de enteros desordenado, implementa el algoritmo Quick Sort donde el pivote es siempre el primer elemento.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1